FINAL PROJECT: AI-Powered Smart Hiring & Candidate Intelligence Platform

---

🧩 📌 Business Scenario

A fast-growing company **"TalentAI Solutions"** receives **1000+ job applications daily**.

Current problems:

- Recruiters manually screen resumes → very slow
- No proper candidate ranking system
- Poor matching between job description & candidate skills
- No analytics on hiring trends
- No AI assistant for recruiters
- No automation in candidate evaluation

👉 The company wants an **AI-powered system** that:

- Automatically analyzes resumes
- Matches candidates with job roles
- Predicts candidate success probability
- Provides insights & dashboards
- Includes a chatbot for recruiter queries

🎯 🎯 PROJECT OBJECTIVE

Build a **full-stack AI/ML + GenAI system** that:

✓ Processes resumes (PDF/CSV/Text)
✓ Performs ML-based candidate scoring
✓ Uses NLP for skill extraction
✓ Implements RAG-based chatbot
✓ Provides dashboards & insights
✓ Deploys as a web app (Streamlit / API)

🧱 🔷 FUNCTIONAL REQUIREMENTS

🔹 1. Data Handling & Preprocessing

Dataset:

- Resume dataset (CSV or text)
- Job descriptions dataset

Tasks:

- Load data using **Pandas**
- Clean:
  - Missing values
  - Duplicate entries
- Feature engineering:
  - Experience (years)
  - Skills count
- Encode categorical data
- Scale numerical features

🔹 2. NLP Processing (Core)

Resume Processing:

- Tokenization
- Stopword removal
- Lemmatization

Extract:

- Skills
- Education
- Experience

Feature Engineering:

- TF-IDF / Bag of Words
- N-grams

👉 Output:

- Structured resume features

🔹 3. Candidate Matching Engine

Build a system to:

- Compare:
  - Resume vs Job Description
- Use:
  - Cosine similarity
  - TF-IDF vectors

👉 Output:

- Match score (0–100)

🔹 4. Machine Learning Model

Problem:

Predict:
👉 "Will candidate be shortlisted?" (Yes/No)

Models to try:

- Logistic Regression
- Decision Tree
- Random Forest
- SVM

Tasks:

- Train-test split
- Cross-validation
- Hyperparameter tuning (GridSearchCV)

🔹 **5. Model Evaluation**

Evaluate using:

- Accuracy
- Precision / Recall
- F1-score
- ROC-AUC

👉 Compare models and select best one

🔹 **6. Feature Engineering (Advanced)**

- Create:
  - Skill match score
  - Experience weight
  - Domain relevance

👉 Improve model performance

🔹 **7. Unsupervised Learning (Bonus Insight)**

- Cluster candidates using:
  - K-Means
- Find:
  - Candidate segments

🔹 8. Dimensionality Reduction

- Use PCA:
  - Reduce features
  - Visualize candidate clusters

🔹 9. Model Saving

- Save model using:
  - Pickle / Joblib

🔹 10. Streamlit Web App

Build UI with:

Features:

- Upload resume
- Enter job description
- Show:
  - Match score
  - Prediction (Shortlist/Reject)
  - Extracted skills

🔹 11. Generative AI Integration (IMPORTANT)

Build an AI Recruiter Assistant

Use LLM to:

- Summarize resume
- Suggest interview questions
- Generate candidate feedback

🔹 12. Prompt Engineering

Design prompts for:

- Resume summarization
- Skill extraction
- Feedback generation

👉 Show:

- Prompt versions
- Improvements

🔹 13. RAG (Retrieval-Augmented Generation)

Build Chatbot:

- Upload:
  - Company hiring policies
- Use:
  - Embeddings
  - Similarity search

Chatbot answers:

- “What skills are required for this role?”
- “Why was candidate rejected?”

🔹 14. API Development

- Use Flask / FastAPI
- Create endpoints:
  - /predict
  - /match-score
  - /chat

🔹 15. Data Visualization & Dashboard

Use:

- Matplotlib / Seaborn

Show:

- Hiring trends
- Skill demand
- Candidate distribution

🔹 16. Deployment

- Run locally or cloud
- Integrate:
    - ML model + Streamlit + API

🎤 🔥 FINAL DEMO REQUIREMENTS

Each team must demonstrate:

1. Problem Understanding

- Hiring challenges

2. Data Processing

- Cleaning + Feature engineering

3. ML Model

- Training + evaluation

4. NLP System

- Skill extraction

5. GenAI Integration

- Resume summary

- Feedback generation

6. RAG Chatbot

- Live Q&A

7. UI Demo

- Streamlit app

8. API Demo

- JSON response